

Lab Experiments

Experiment 1: Average Sale Price Using NumPy

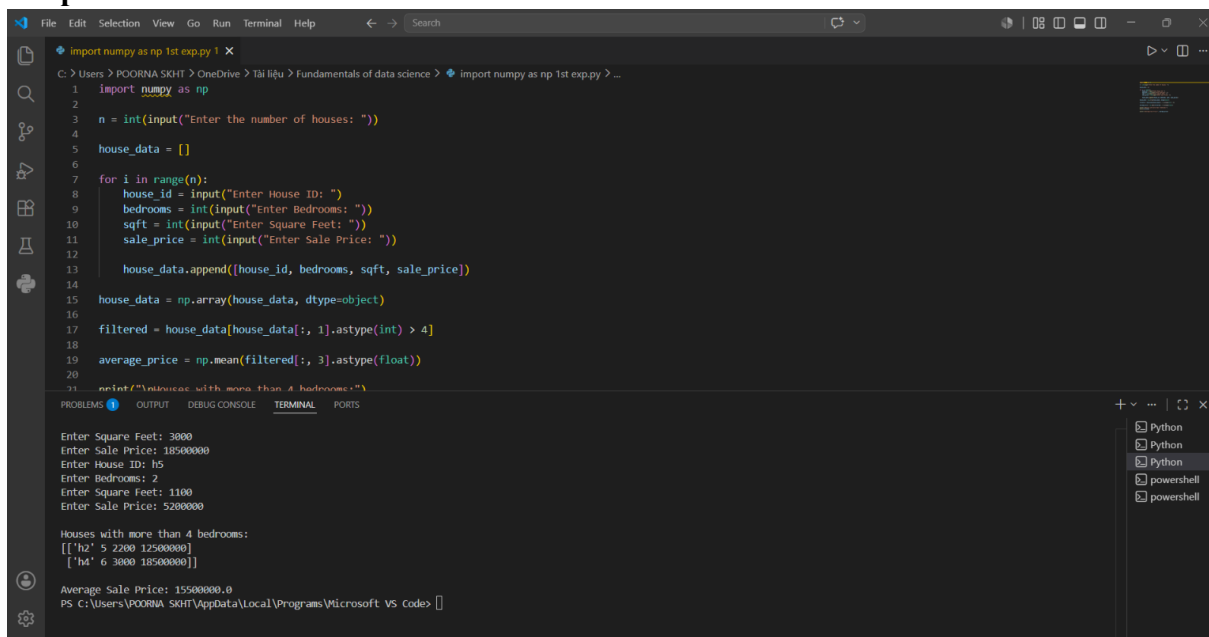
Aim

To analyze the housing dataset using the NumPy library and calculate the average sale price of houses that have more than four bedrooms by applying NumPy array operations and filtering techniques.

Algorithm

1. Import the NumPy library.
2. Create a NumPy array containing House ID, Bedrooms, Square Feet, and Sale Price.
3. Select only those houses where the number of bedrooms is greater than four.
4. Extract the sale price column from the filtered data.
5. Calculate the average sale price using the `np.mean()` function.
6. Display the filtered houses and the average sale price.

Output:



```
import numpy as np

n = int(input("Enter the number of houses: "))
house_data = []

for i in range(n):
    house_id = input("Enter House ID: ")
    bedrooms = int(input("Enter Bedrooms: "))
    sqft = int(input("Enter Square Feet: "))
    sale_price = int(input("Enter Sale Price: "))
    house_data.append([house_id, bedrooms, sqft, sale_price])

house_data = np.array(house_data, dtype=object)
filtered = house_data[house_data[:, 1].astype(int) > 4]
average_price = np.mean(filtered[:, 3].astype(float))

print("Houses with more than 4 bedrooms:")
print(filtered)

print("Average Sale Price: ", average_price)
```

Enter Square Feet: 3000
Enter Sale Price: 18500000
Enter House ID: h5
Enter Bedrooms: 2
Enter Square Feet: 1100
Enter Sale Price: 5200000

Houses with more than 4 bedrooms:
[["h2" 5 2200 12500000]
["h4" 6 3000 18500000]]

Average Sale Price: 15500000.0
PS C:\Users\POORNA SKHT\AppData\Local\Programs\Microsoft VS Code>

Result

The NumPy program successfully filtered the houses having more than four bedrooms and calculated their average sale price. This demonstrates how NumPy can efficiently perform filtering and statistical operations on datasets.

Experiment 2: Email Spam Detection Using Logistic Regression

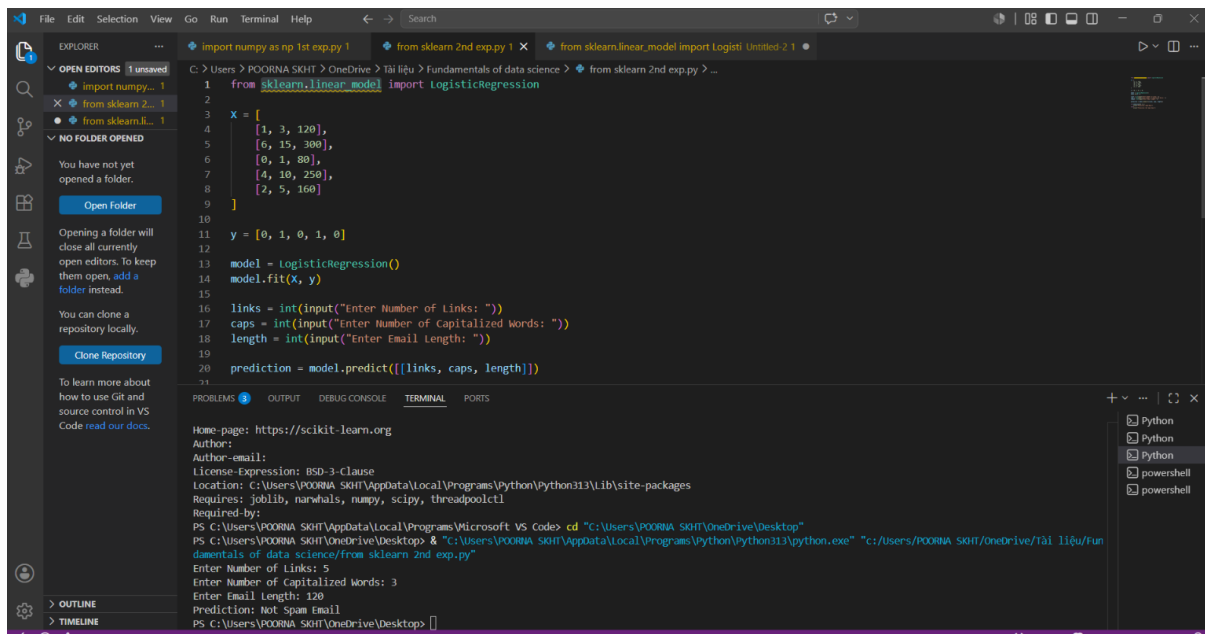
Aim

To build a Logistic Regression model that classifies incoming emails as spam or non-spam based on features such as the number of links, capitalized words, and email length.

Algorithm

1. Import the Logistic Regression model from the Scikit-learn library.
2. Prepare the feature dataset (Number of Links, Capitalized Words, Email Length).
3. Prepare the target dataset (Spam = 1, Not Spam = 0).
4. Train the Logistic Regression model using the training data.
5. Accept feature values for a new email from the user.
6. Predict whether the email is spam or not.
7. Display the prediction result.

Output:



```
1 import numpy as np
2
3 from sklearn.linear_model import LogisticRegression
4
5 X = [
6     [1, 3, 120],
7     [6, 15, 300],
8     [0, 1, 80],
9     [4, 10, 250],
10    [2, 5, 160]
11 ]
12
13 y = [0, 1, 0, 1, 0]
14
15 model = LogisticRegression()
16 model.fit(X, y)
17
18 links = int(input("Enter Number of Links: "))
19 caps = int(input("Enter Number of Capitalized Words: "))
20 length = int(input("Enter Email length: "))
21
22 prediction = model.predict([[links, caps, length]])
23
24 print(prediction)
```

Terminal Output:

```
PS C:\Users\POORNA SKH\OneDrive\Desktop> cd "C:\Users\POORNA SKH\OneDrive\Desktop"
PS C:\Users\POORNA SKH\OneDrive\Desktop> & "C:\Users\POORNA SKH\AppData\Local\Programs\Python\Python313\python.exe" "C:\Users\POORNA SKH\OneDrive\Tài liệu\Fundamentals of data science\from sklearn 2nd exp.py"
Enter Number of Links: 5
Enter Number of Capitalized Words: 3
Enter Email length: 120
Prediction: Not Spam Email
PS C:\Users\POORNA SKH\OneDrive\Desktop>
```

Result

The Logistic Regression model was trained successfully and accurately predicted whether a newly received email is spam or non-spam based on the given input features.

Experiment 3: Email Classification Using Logistic Regression

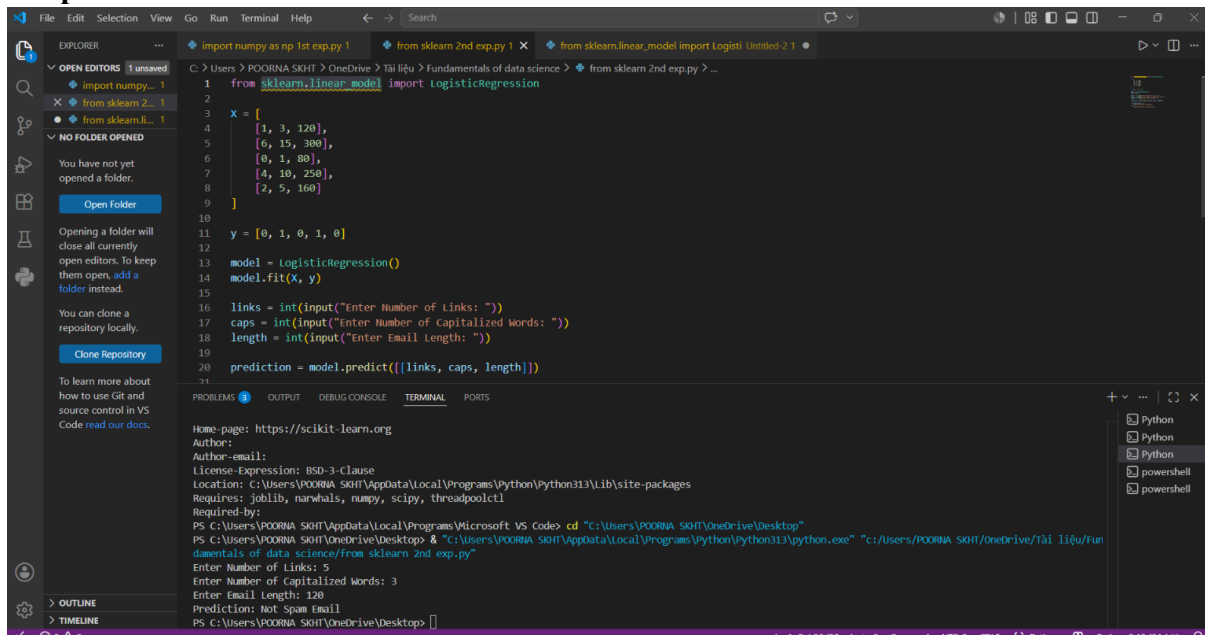
Aim

To implement a machine learning classification model using Logistic Regression for identifying spam emails from historical email data.

Algorithm

1. Import the required machine learning libraries.
2. Create the training dataset with email features and labels.
3. Train the Logistic Regression classifier.
4. Accept input values for a new email.
5. Use the trained model to classify the email.
6. Display the predicted email category.

Output:



```
1 import numpy as np
2 from sklearn.linear_model import LogisticRegression
3
4 X = [
5     [1, 3, 120],
6     [6, 15, 300],
7     [0, 1, 80],
8     [4, 10, 250],
9     [2, 5, 160]
10 ]
11
12 y = [0, 1, 0, 1, 0]
13
14 model = LogisticRegression()
15 model.fit(X, y)
16
17 links = int(input("Enter Number of Links: "))
18 caps = int(input("Enter Number of Capitalized Words: "))
19 length = int(input("Enter Email Length: "))
20
21 prediction = model.predict([[links, caps, length]])
```

Home-page: <https://scikit-learn.org>
Author:
Author-email:
License-Expression: BSD-3-Clause
Location: C:\Users\POORNA SIKH\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires: joblib, narwhals, numpy, scipy, threadpoolctl
Required-by:
PS C:\Users\POORNA SIKH\AppData\Local\Programs\Microsoft VS Code> cd "C:\Users\POORNA SIKH\OneDrive\Desktop"
PS C:\Users\POORNA SIKH\OneDrive\Desktop> & "C:\Users\POORNA SIKH\AppData\Local\Programs\Python\Python313\python.exe" "C:\Users\POORNA SIKH\OneDrive\Tài liệu\Fundamentals of data science\from sklearn 2nd exp.py"
Enter Number of Links: 5
Enter Number of Capitalized Words: 3
Enter Email Length: 120
Prediction: Not Spam Email
PS C:\Users\POORNA SIKH\OneDrive\Desktop>

Result

The Logistic Regression classifier successfully classified the new email as either spam or not spam. This experiment demonstrates the use of supervised learning for email filtering applications.

Experiment 4: Car Resale Price Prediction Using Linear Regression

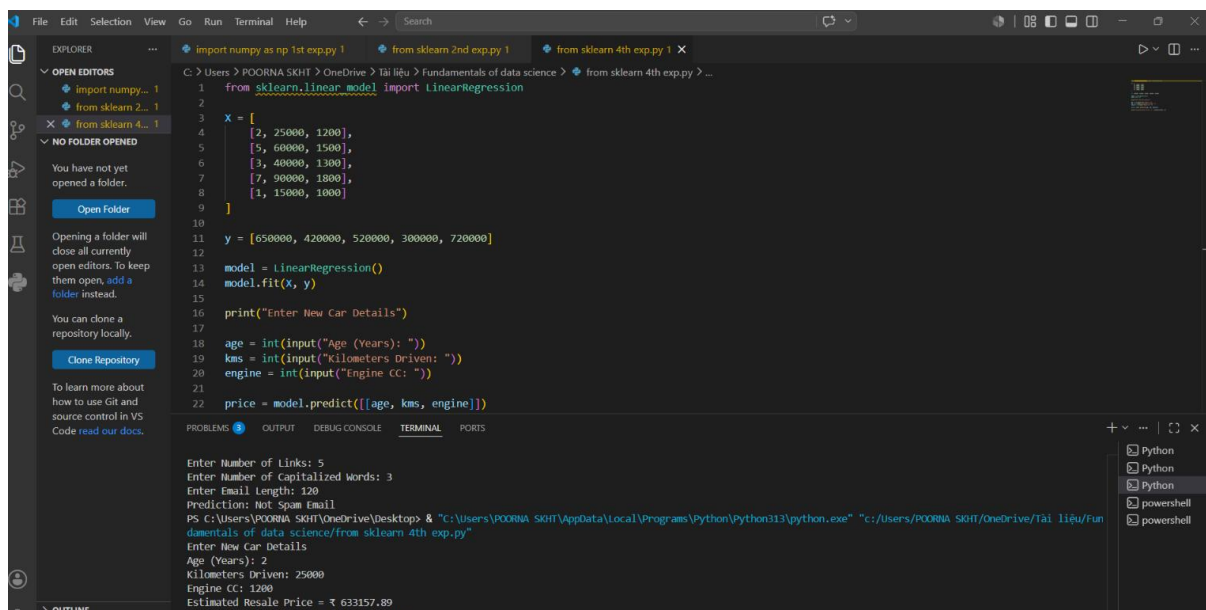
Aim

To develop a Linear Regression model that predicts the resale price of a used car based on its age, kilometers driven, and engine capacity.

Algorithm

1. Import the Linear Regression model.
2. Prepare the dataset containing car features and resale prices.
3. Train the Linear Regression model.
4. Accept user input for a new car.
5. Predict the estimated resale price using the trained model.
6. Display the predicted resale price.

Output:



```
File Edit Selection View Go Run Terminal Help
import numpy as np
from sklearn.linear_model import LinearRegression

X = [
    [2, 25000, 1200],
    [5, 40000, 1500],
    [3, 40000, 1300],
    [7, 90000, 1800],
    [1, 15000, 1000]
]

y = [650000, 420000, 520000, 300000, 720000]

model = LinearRegression()
model.fit(X, y)

print("Enter New Car Details")

age = int(input("Age (Years): "))
kms = int(input("Kilometers Driven: "))
engine = int(input("Engine CC: "))

price = model.predict([[age, kms, engine]])

Enter Number of Links: 5
Enter Number of Capitalized Words: 3
Enter Email Length: 120
Prediction: Not Spam Email
PS C:\Users\POORNA SIKHI\OneDrive\Desktop> "C:\Users\POORNA SIKHI\AppData\Local\Programs\Python\Python313\python.exe" "c:/Users/POORNA SIKHI/OneDrive/Tài liệu/Fundamentals of data science/from sklearn 4th exp.py"
Enter New Car Details
Age (Years): 2
Kilometers Driven: 25000
Engine CC: 1200
Estimated Resale Price = ₹ 633157.89
```

Result

The Linear Regression model successfully estimated the resale price of the given car based on the input features. The experiment demonstrates the application of regression techniques for predicting continuous numerical values.

Experiment 5: Loan Approval Prediction Using CART Decision Tree

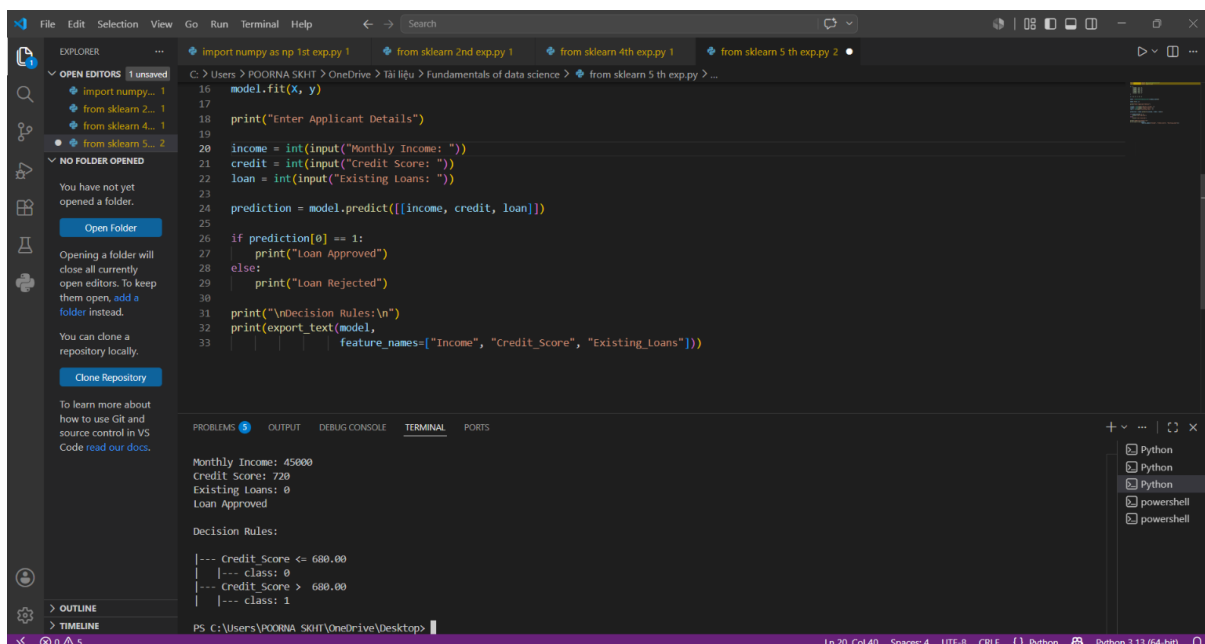
Aim

To build a Decision Tree Classifier using the CART algorithm for predicting whether a loan application should be approved or rejected based on applicant details.

Algorithm

1. Import the Decision Tree Classifier from the Scikit-learn library.
2. Prepare the training dataset with applicant information.
3. Train the CART Decision Tree model.
4. Accept details of a new applicant.
5. Predict whether the loan should be approved or rejected.
6. Display the prediction and the decision rules generated by the model.

Output:



```
16 model.fit(x, y)
17
18 print("Enter Applicant Details")
19
20 income = int(input("Monthly Income: "))
21 credit = int(input("Credit Score: "))
22 loan = int(input("Existing Loans: "))
23
24 prediction = model.predict([income, credit, loan])
25
26 if prediction[0] == 1:
27     print("Loan Approved")
28 else:
29     print("Loan Rejected")
30
31 print("\nDecision Rules:\n")
32 print(export_text(model,
33                  feature_names=["Income", "Credit_Score", "Existing_Loans"]))
```

Monthly Income: 45000
Credit Score: 720
Existing Loans: 0
Loan Approved

Decision Rules:

```
|-> Credit_Score <= 680.00
|   |-> class: 0
|   |-> Credit_Score > 680.00
|       |-> class: 1
```

Result

The CART Decision Tree model successfully classified the loan application as approved or rejected based on the applicant's income, credit score, and existing loans. The generated decision rules provide a clear explanation of the classification process.